# OWASP Top 10 2025 - Application Design Flaws

## Security Misconfigurations

Security misconfigurations happen when systems, servers, or applications are deployed with unsafe defaults, incomplete settings, or exposed services

small misconfigurations can expose sensitive data, enable privilege escalation, or give attackers a foothold into the system.

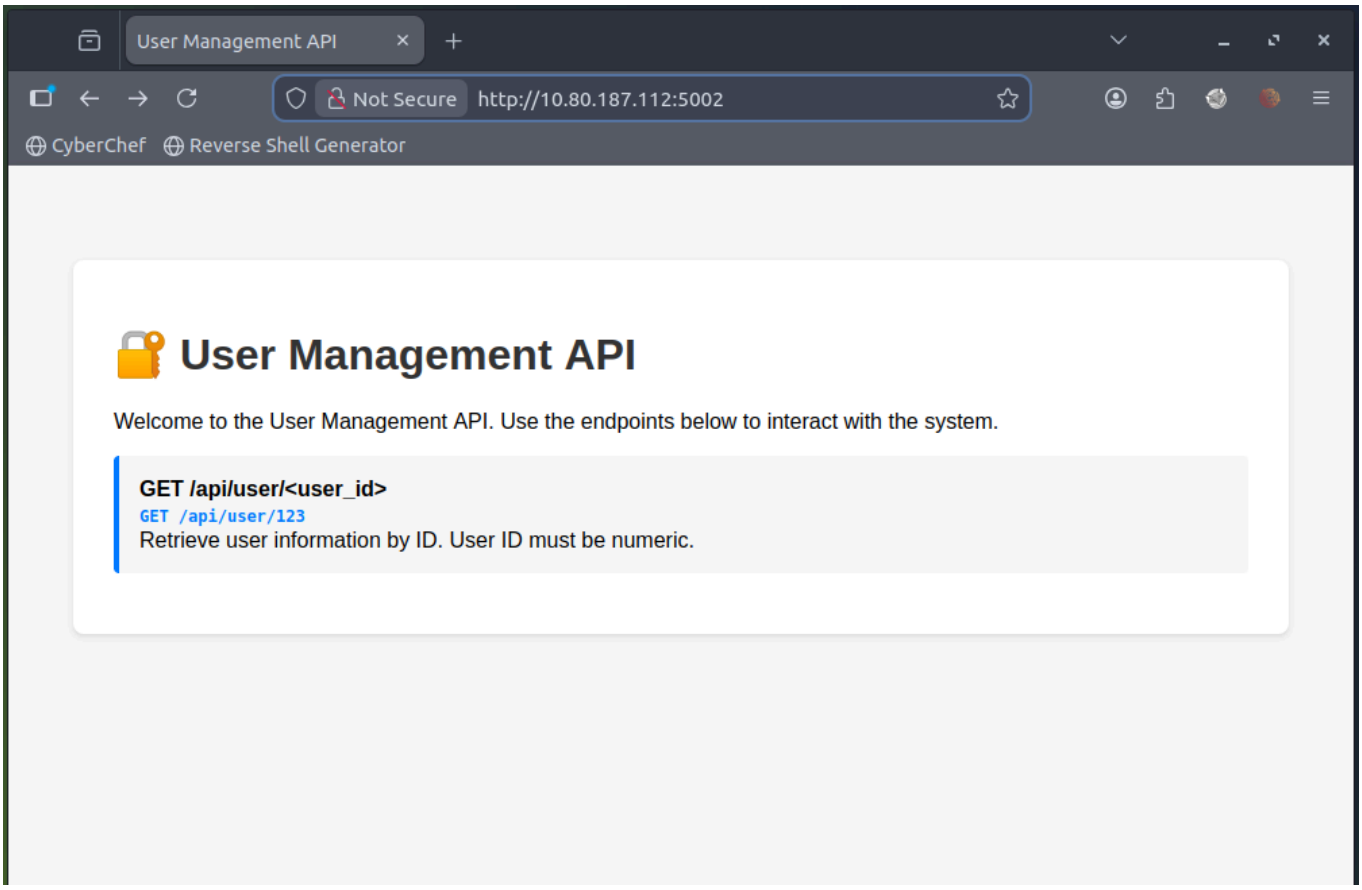
**Common Patterns**

- Default credentials or weak passwords left unchanged
- Unnecessary services or endpoints exposed to the internet
- Misconfigured cloud storage or permissions (S3, Azure Blob, GCP buckets)
- Unrestricted API access or missing authentication/authorisation
- Verbose error messages exposing stack traces or system details
- Outdated software, frameworks, or containers with known vulnerabilities
- Exposed AI/ML endpoints without proper access controls

**Common Patterns**

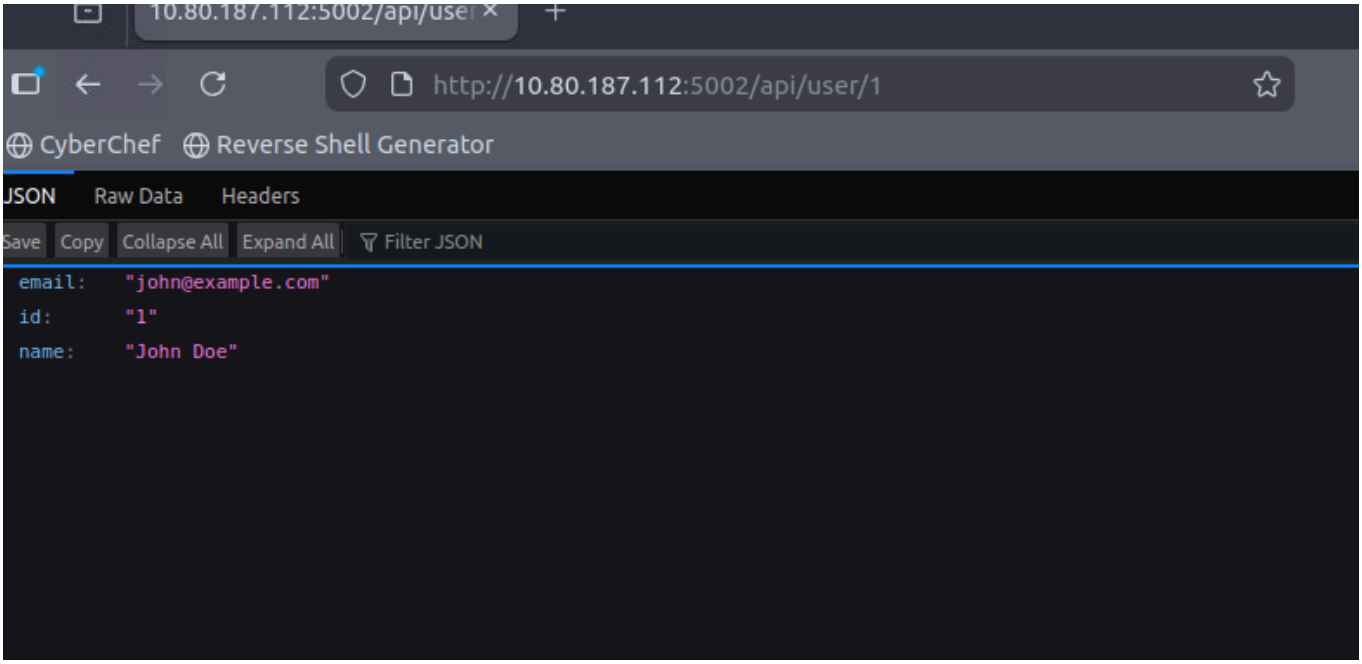
- Default credentials or weak passwords left unchanged
- Unnecessary services or endpoints exposed to the internet
- Misconfigured cloud storage or permissions (S3, Azure Blob, GCP buckets)
- Unrestricted API access or missing authentication/authorisation
- Verbose error messages exposing stack traces or system details
- Outdated software, frameworks, or containers with known vulnerabilities
- Exposed AI/ML endpoints without proper access controls

## Practical Example

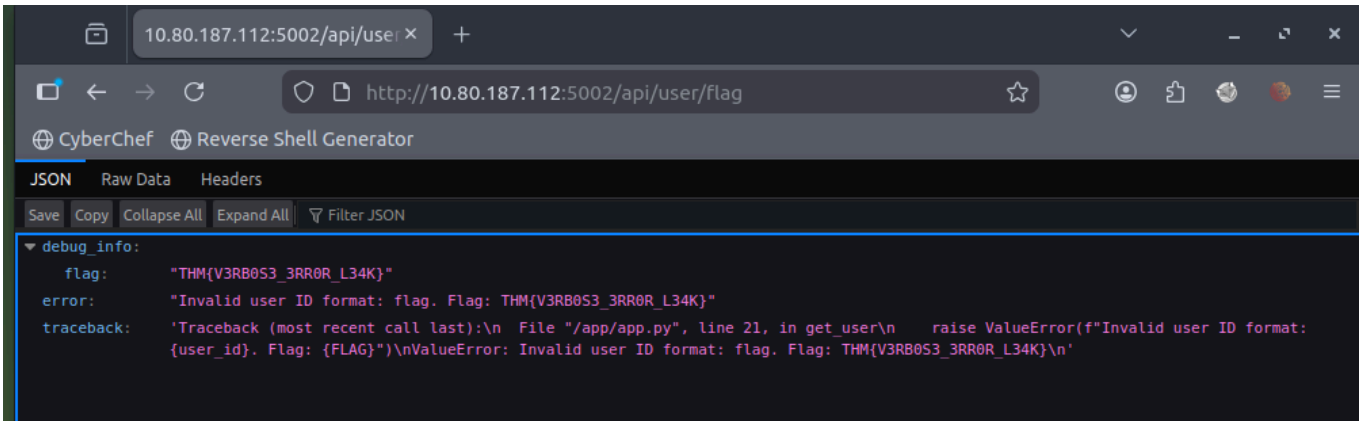



This tells me how i can exploit the URL to get user information

for example:




i then tried flag id to get the flag and it provided it to me




# Software Supply Chain Failures

### What It Is

Software supply chain failures happen when applications rely on components, libraries, services, or models that are compromised, outdated, or improperly verified. These weaknesses are not inherent in your code, but rather in the software and tools you depend on. Attackers exploit these weak links to inject malicious code, bypass security, or steal sensitive data.

### Why It Matters

Modern applications are built from many third-party packages, APIs, and AI models. One compromised dependency can compromise your entire system, allowing attackers to gain access without ever touching your own code.

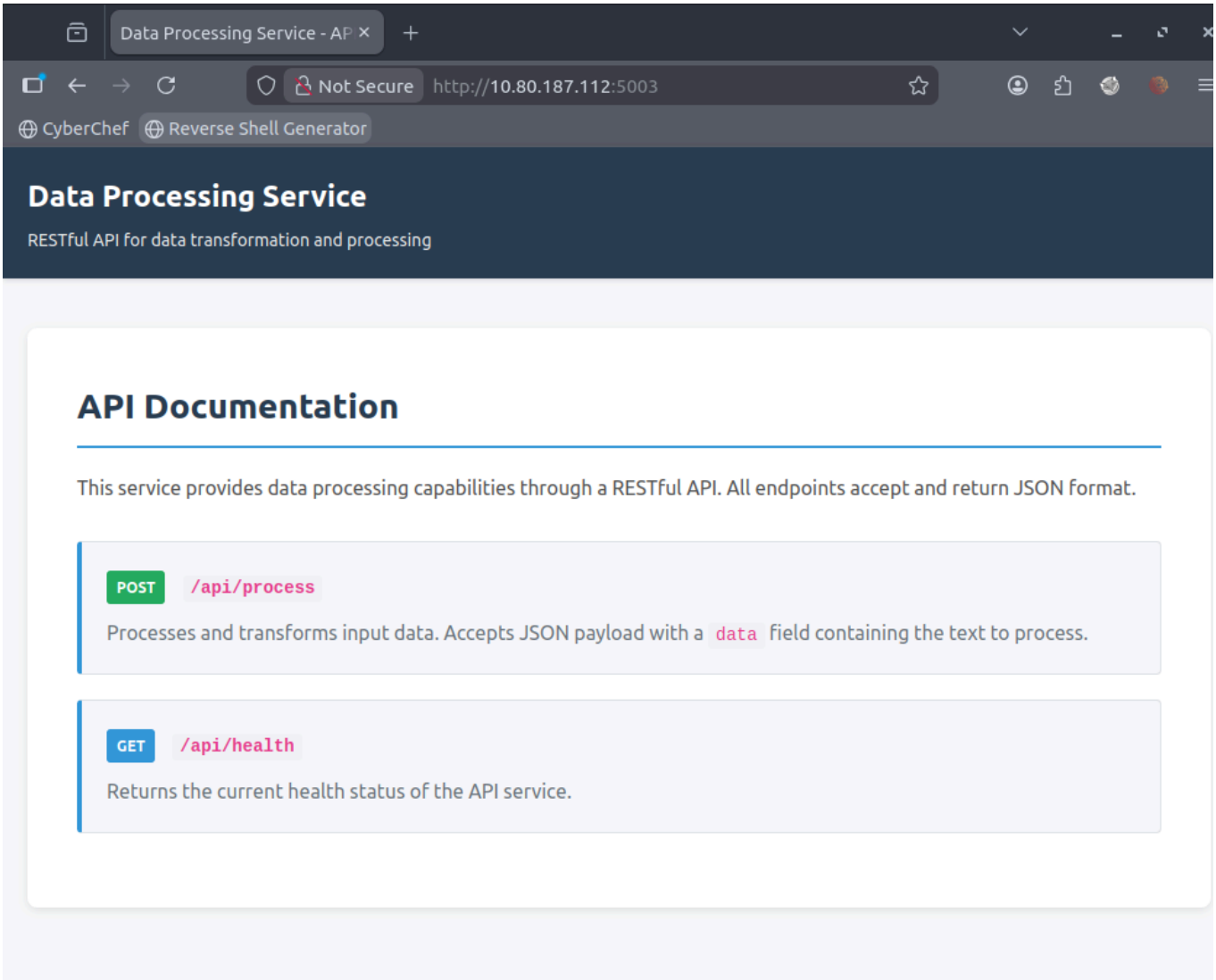
### Common Patterns

- Using unverified or unmaintained libraries and dependencies
- Automatically installing updates without verification
- Over-reliance on third-party AI models without monitoring or auditing
- Insecure build pipelines or CI/CD processes that allow tampering
- Poor license or provenance tracking for components
- Lack of monitoring for vulnerabilities in dependencies after deployment

### How To Protect The Supply Chain

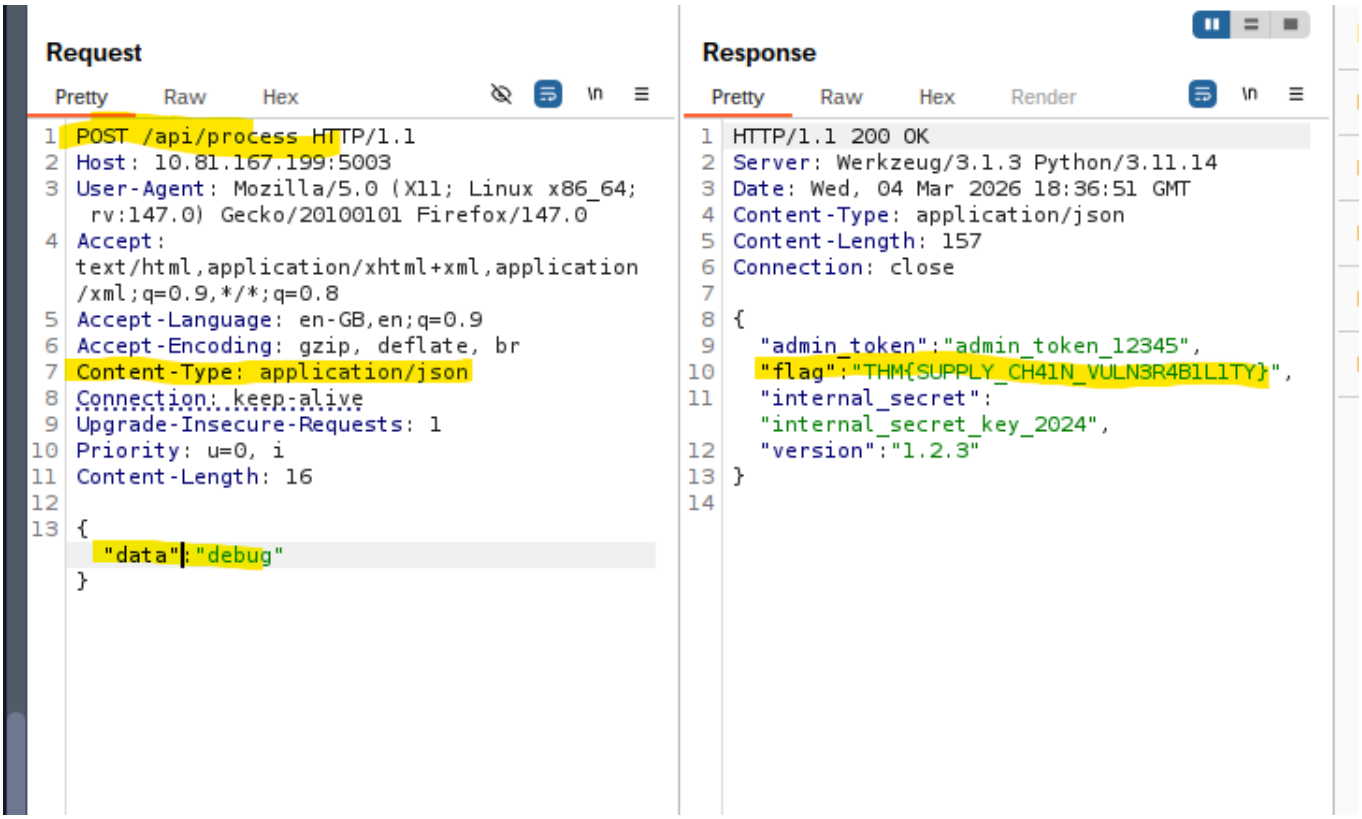
- Verify all third-party components, libraries, and AI models before use
- Monitor and patch dependencies regularly
- Sign, verify, and audit software updates and packages
- Lock down CI/CD pipelines and build processes to prevent tampering
- Track provenance and licensing for all dependencies
- Implement runtime monitoring for unusual behaviour from dependencies or AI components
- Integrate supply chain threat modelling into the SDLC, including testing, deployment, and update workflows

## Practical Example




I opened up burpsuite for this task and captured the get request for /api/process page
I then edited it from a GET request to a POST request

From here I added content type and then the json snipped and sent that request and it gave me the flag.




## Cryptographic Failures

Cryptographic failures happen when encryption is used incorrectly or not at all. This includes weak algorithms, hard-coded keys, poor key handling, or unencrypted sensitive data

### Why It Matters

Web applications rely on cryptography everywhere: protecting network traffic, securing stored data, verifying identities, and safeguarding secrets. When these controls fail, sensitive data such as passwords, tokens, or personal information can be exposed, leading to account takeovers or full-scale breaches.

Attackers can exploit these flaws through man-in-the-middle attacks, brute-force attacks on weak keys, or by simply discovering secrets that were never properly protected.

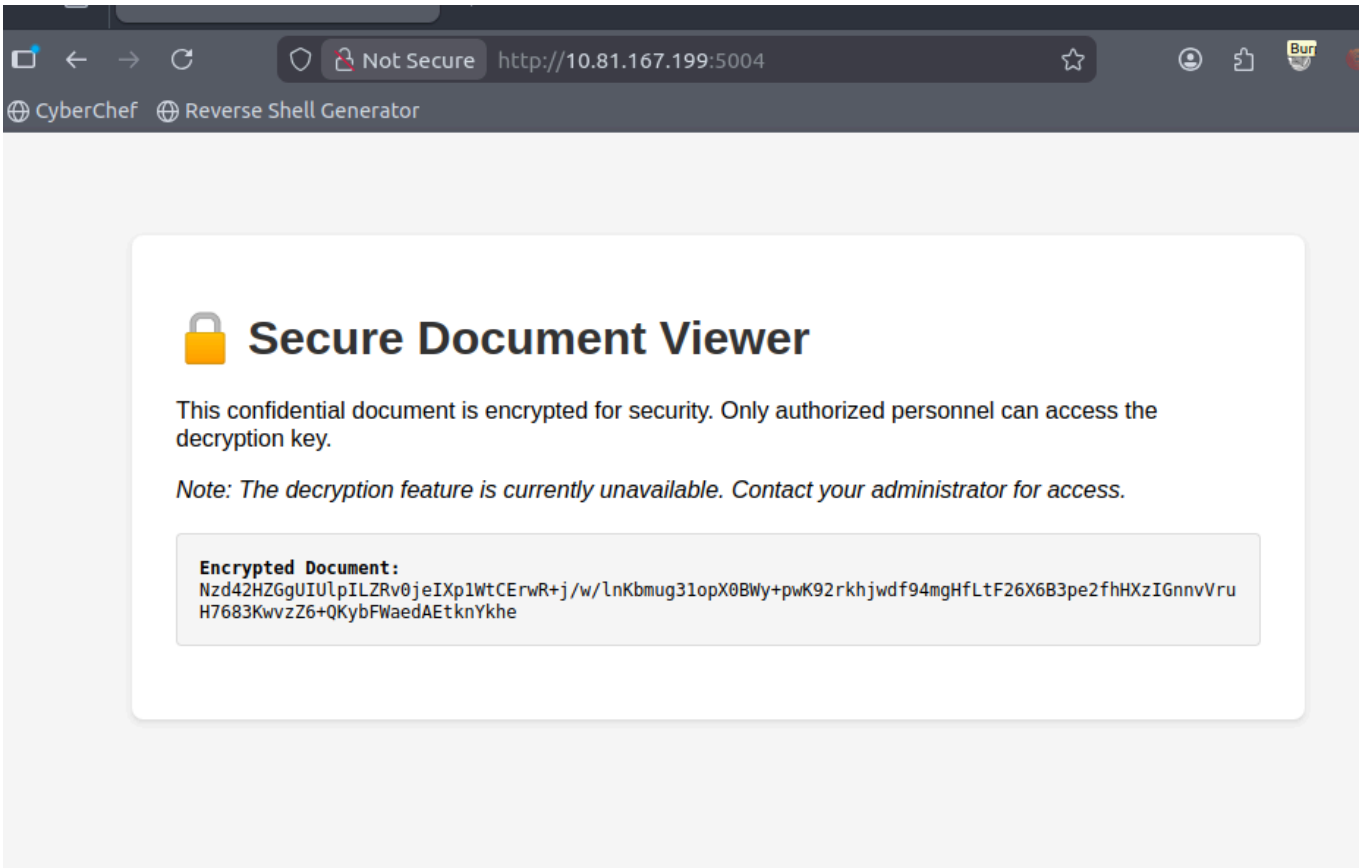
### Common Patterns

- Using deprecated or weak algorithms like MD5, SHA-1, or ECB mode
- Hard-coded secrets in code or configuration
- Poor key rotation or management practices
- Lack of encryption for sensitive data at rest or in transit
- Self-signed or invalid TLS certificates
- Using AI/ML systems without proper secret handling for model parameters or sensitive inputs

### How To Prevent It

- Use strong, modern algorithms such as AES-GCM, ChaCha20-Poly1305, or enforce TLS 1.3 with valid certificates
- Use secure key management services like Azure Key Vault, AWS KMS, or HashiCorp Vault
- Rotate secrets and keys regularly, following defined crypto periods
- Document and enforce policies and standard operating procedures for key lifecycle management
- Maintain a complete inventory of certificates, keys, and their owners
- Ensure AI models and automation agents never expose unencrypted secrets or sensitive data
- The web application in this room contains a weakness of this type for you to explore.

## Practical Task:




I went to the pages source and found a decrypt.js script and clicked on it

Not Secure view-source:http://10.81.167.199:5004/

CyberChef Reverse Shell Generator

```
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Secure Document Viewer</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            max-width: 800px;
            margin: 50px auto;
            padding: 20px;
            background: #f5f5f5;
        }
        .container {
            background: white;
            padding: 30px;
            border-radius: 8px;
            box-shadow: 0 2px 4px rgba(0,0,0,0.1);
        }
        h1 {
            color: #333;
        }
        .encrypted-box {
            background: #f9f9f9;
            border: 1px solid #ddd;
            padding: 15px;
            border-radius: 4px;
            font-family: monospace;
            word-break: break-all;
            margin: 20px 0;
        }
    </style>
</head>
<body>
    <div class="container">
        <h1>🔒 Secure Document Viewer</h1>
        <p>This confidential document is encrypted for security. Only authorized personnel can access the decryption key.</p>
        <p><em>Note: The decryption feature is currently unavailable. Contact your administrator for access.</em></p>

        <div class="encrypted-box" id="encrypted">
            <strong>Encrypted Document:</strong><br>
            Nzd42HZGgUIUlpILZRv0jeIXp1WtCErwR+j/w/lnKbmug31opX0BWy+pwK92rkhjwdf94mgHfLtF26X6B3pe2fhHXzIGnnvVruH7683KwvzZ6+QKybFWa
        </div>
    </div>

    <script src="/static/js/decrypt.js"></script>
</body>
```

Here is the decrypt.js script:

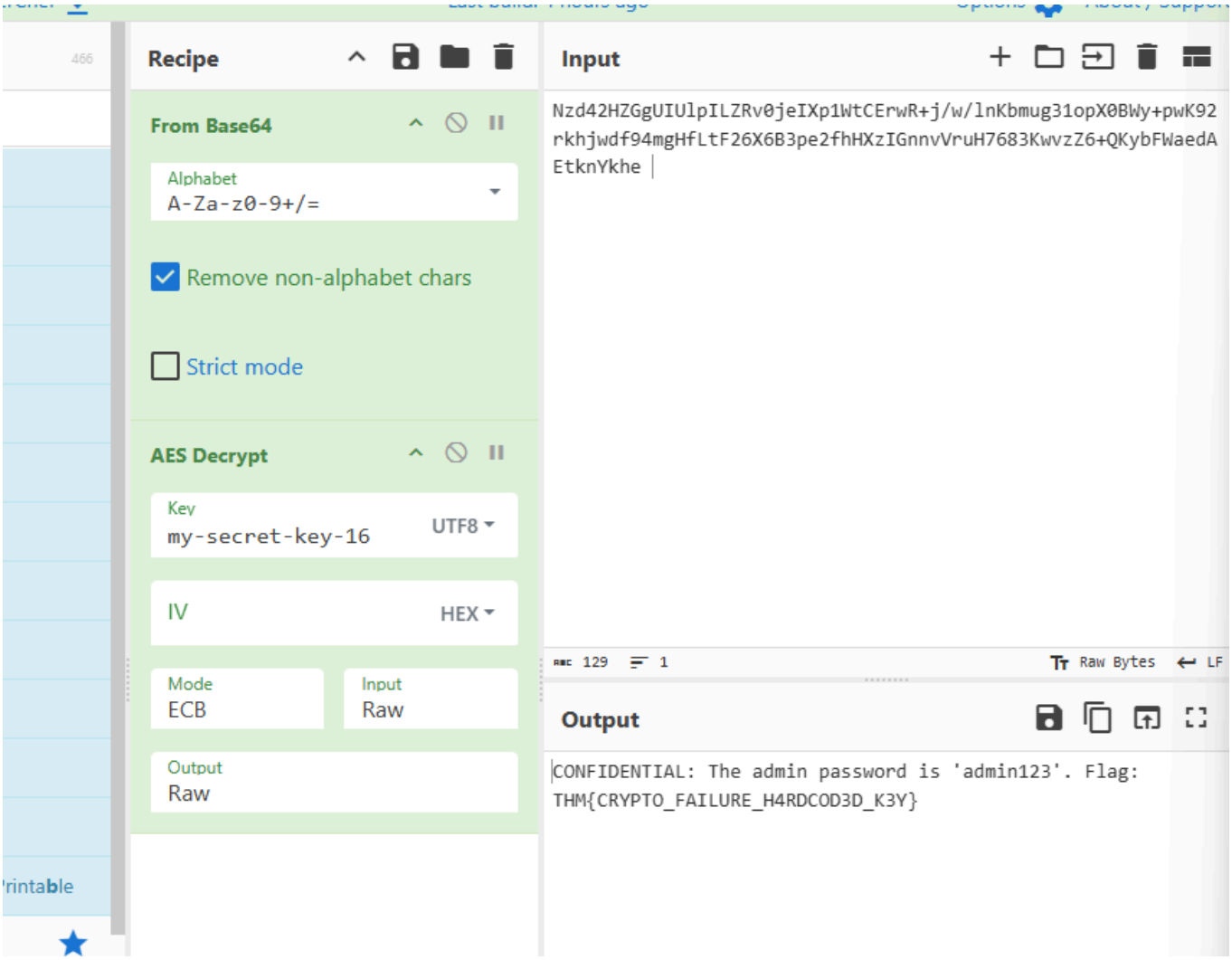
Not Secure view-source:http://10.81.167.199:5004/static/js/decrypt.js

CyberChef Reverse Shell Generator

```
// Document encryption utilities
// Last updated: 2024-03-15

(function() {
    'use strict';

    // Configuration
    const SECRET_KEY = "my-secret-key-16";
    const ENCRYPTION_MODE = "ECB";
    const KEY_SIZE = 128;

    // Utility functions
    function formatTimestamp(date) {
        return date.toISOString().replace('T', ' ').substring(0, 19);
    }

    function validateDocumentId(id) {
        return /^[a-zA-Z0-9-_]+$/.test(id);
    }

    function getDocumentMetadata() {
        return {
            version: "1.2.3",
            encryptionEnabled: false,
            lastModified: formatTimestamp(new Date())
        };
    }

    // Logging utility
    function logEvent(eventType, details) {
        if (typeof console !== 'undefined' && console.log) {
            console.log(`[${formatTimestamp(new Date())}] ${eventType}:`, details);
        }
    }

    // Document validation
    function isValidDocumentFormat(data) {
        try {
            const decoded = atob(data);
            return decoded.length > 0 && decoded.length % 16 === 0;
        } catch (e) {
            return false;
        }
    }
```

it shows the encryption key that i need to use to decrypt the message on the main page

I then went in cyber chef to decrypt the message




## Insecure Design

Insecure design happens when flawed logic or architecture is built into a system from the start. These flaws stem from skipped threat modelling, no design requirements or reviews, or accidental errors.

### Why It Matters

You can't patch an insecure design. It's built into the workflow, logic, and trust boundaries. Fixing it means rethinking how systems, and now AI, make decisions.

### Common Insecure Designs In 2025

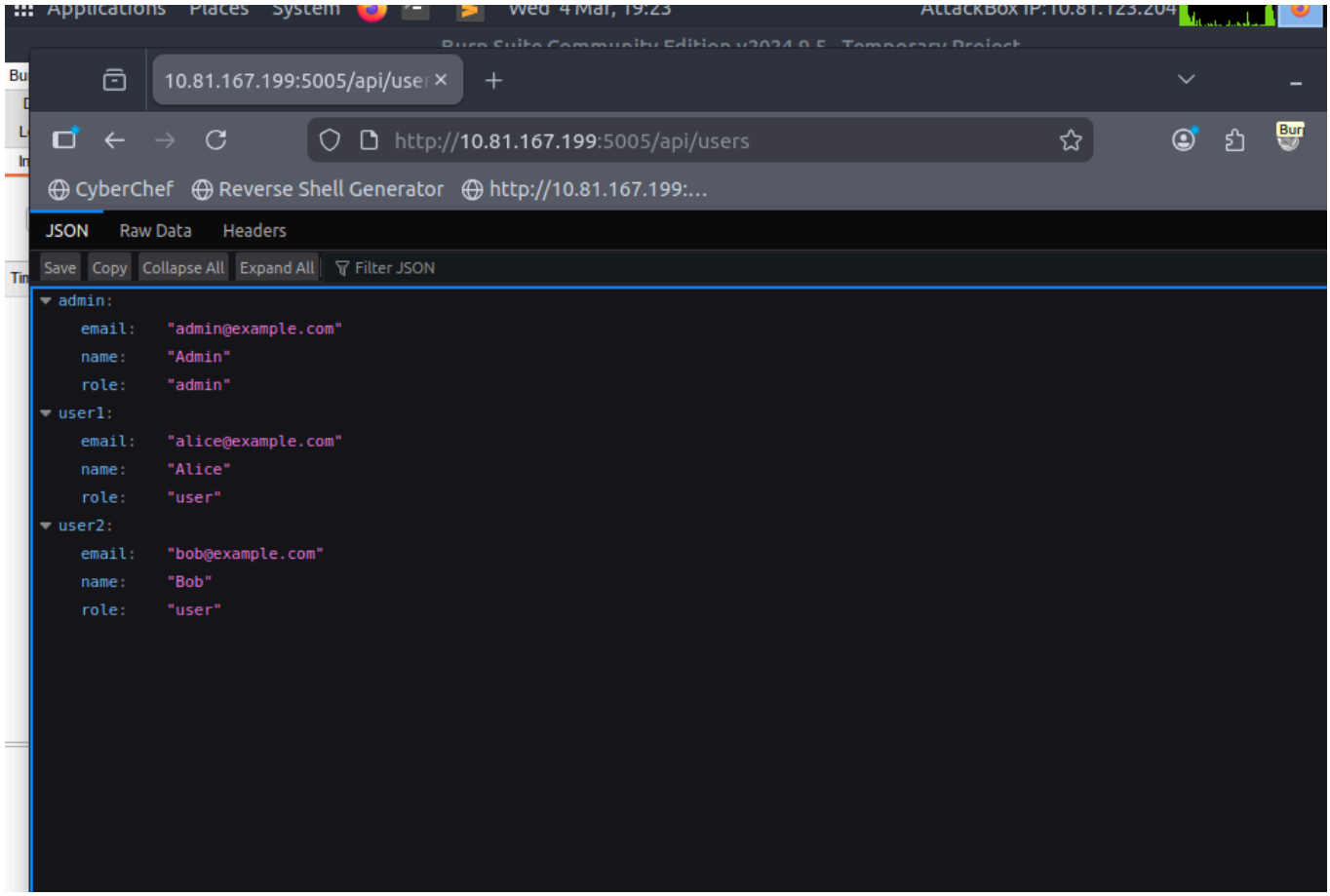
- Weak business logic controls, like recovery or approval flows
- Flawed assumptions about user or model behaviour
- AI components with unchecked authority or access
- Missing guardrails for LLMs and automation agents
- Test or debug bypasses left in production
- No consistent abuse-case review or AI threat modelling

### Insecure Design In The AI Era

AI introduces new kinds of design failures. For example, prompt injection occurs when user input is blended with system prompts, allowing attackers to hijack the context or extract hidden data.

### How To Design Securely

- Treat every model as untrusted until proven otherwise.
- Validate and filter all model inputs and outputs to ensure accuracy and integrity.
- Separate system prompts from user content.
- Keep sensitive data out of prompts unless absolutely needed and protect it with strict controls.
- Require human review for high-risk AI actions.
- Log model provenance, monitor behaviour, and apply differential privacy for sensitive data.
- Include AI-specific threat modelling for prompt attacks, inference risks, agent misuse, and supply chain compromise throughout the design process.
- Build threat modelling into every stage of development, not just at the start.
- Define clear security requirements for each feature before implementation.
- Apply the principle of least privilege across users, APIs, and services.
- Ensure proper authentication, authorisation, and session management across the system.
- Keep dependencies, third-party components, and supply chain sources verified and up to date.
- Continuously monitor and test the system for logic flaws, abuse paths, and emergent risks as new features or AI components are added.


Applications Places System
Wed 4 Mar, 19:23
AttackBox IP: 10.81.123.204
10.81.167.199:5005/api/user
http://10.81.167.199:5005/api/users
CyberChef
Reverse Shell Generator
http://10.81.167.199:...
JSON
Raw Data
Headers
Save
Copy
Collapse All
Expand All
Filter JSON
admin:
email: "admin@example.com"
name: "Admin"
role: "admin"
user1:
email: "alice@example.com"
name: "Alice"
role: "user"
user2:
email: "bob@example.com"
name: "Bob"
role: "user"

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>SecureChat - Mobile App</title>
    <style>
        * {
            margin: 0;
            padding: 0;
            box-sizing: border-box;
        }
        body {
            font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, Oxygen, Ubuntu, Cantarell, sans-serif;
            background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
            min-height: 100vh;
            display: flex;
            align-items: center;
            justify-content: center;
            padding: 20px;
        }
        .container {
            background: white;
            border-radius: 20px;
            box-shadow: 0 20px 60px rgba(0,0,0,0.3);
            padding: 40px;
            max-width: 500px;
            width: 100%;
            text-align: center;
        }
        .logo {
            width: 80px;
            height: 80px;
            background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
            border-radius: 20px;
            margin: 0 auto 20px;
            display: flex;
            align-items: center;
            justify-content: center;
            font-size: 36px;
        }
        h1 {
            color: #2c3e50;
            margin-bottom: 10px;
            font-size: 32px;
        }
        .subtitle {
```

Not Secure view-source:http://10.81.167.199:5005/api/messages/adm

CyberChef Reverse Shell Generator http://10.81.167.199:...


```json
{
  "messages": [
    {
      "content": "Admin panel access key: THM{1NS3CUR3_D35IGN_4SSUMPT10N}",
      "from": "system"
    }
  ],
  "user": "admin"
}
```